PUC-RIO

Fabiano Carvalho Costa Júnior

Relatório de Software Básico
Laboratório: Código de Máquina

Rio de Janeiro

2026

1 Introdução e Configuração do Ambiente

Este laboratório explorou a relação entre o código *assembly*, o código de máquina gerado pelo montador e a execução desse código a partir de dados em memória, para a arquitetura x86-64. A ideia central é que uma função nada mais é do que uma sequência de bytes em memória: se o endereço inicial dessa sequência for convertido para um ponteiro de função, ela pode ser chamada como uma função C comum.

As ferramentas e opções utilizadas foram:

- `gcc -c -o foo.o foo.s`: traduz o arquivo *assembly* para um arquivo objeto (código de máquina).
- `objdump -d foo.o`: realiza o *disassembly* do arquivo objeto, exibindo os bytes de cada instrução em hexadecimal.
- `-Wa,-execstack`: sinaliza que o programa pode executar código armazenado em regiões de dados e pilha.
- `execpage`: função fornecida no enunciado que utiliza `mprotect` para tornar executáveis as páginas de memória que contêm o *array* de bytes.

Em todos os testes, o programa foi compilado com `gcc -Wall -o seuprograma main.c` e a função gerada era chamada com o argumento 10, esperando-se o resultado 11.

2 Função Simples (`foo` $= x + 1$)

A primeira etapa consistiu em traduzir manualmente para *assembly* a função:

```c
int foo (int x) {
  return x+1;
}
```

2.1 Código em Assembly (`foo.s`)

Pela convenção de chamada System V AMD64 ABI, o primeiro argumento inteiro é recebido em `%edi` e o valor de retorno deve ser deixado em `%eax`.

```
.text
.globl foo
foo:
    addl $1, %edi
    movl %edi, %eax
    ret
```

2.2 Código de Máquina (`objdump -d foo.o`)

```
0000000000000000 <foo>:
   0:   83 c7 01        add    $0x1,%edi
   3:   89 f8           mov    %edi,%eax
   5:   c3              ret
```

A função ocupa 6 bytes: `83 c7 01 89 f8 c3`.

2.3 Execução a partir de um *array* de bytes

Os bytes foram copiados para um *array* `unsigned char codigo1[]`, declarado como variável local de `main`. Em seguida, o endereço do *array* foi convertido para um ponteiro de função do tipo `funcp`, definido como "ponteiro para função que recebe um `int` e retorna um `int`".

```c
typedef int (*funcp) (int x);

unsigned char codigo1[] = {0x83, 0xc7, 0x01, 0x89, 0xf8, 0xc3};
execpage(codigo1, sizeof codigo1);
funcp f1 = (funcp)codigo1;
i = (*f1)(10);
printf("Parte 1 (x+1): foo(10) = %d\n", i);
```

A saída obtida foi `foo(10) = 11`, confirmando que o código de máquina contido no *array* foi de fato executado.

3 Chamada de Função (`call add`)

A segunda etapa traduziu a função:

```c
int foo (int x) {
  return add(x);
}
```

3.1 Código em Assembly (`foo_call.s`)

```
.text
.globl foo
foo:
    pushq %rbp
    movq %rsp, %rbp
    call add
    leave
    ret
```

O quadro de pilha (`pushq`/`movq`/`leave`) foi mantido para que a pilha permaneça alinhada em 16 bytes no momento do `call`. Como `%edi` já contém o argumento, ele é repassado diretamente a `add`.

3.2 Código de Máquina e Deslocamento

```
0000000000000000 <foo>:
   0:   55                      push   %rbp
   1:   48 89 e5                mov    %rsp,%rbp
   4:   e8 00 00 00 00          call   9 <foo+0x9>
   9:   c9                      leave
   a:   c3                      ret
```

O byte `e8` é o *opcode* de `call`, e os quatro bytes seguintes guardam o deslocamento (*displacement*) relativo à instrução **seguinte** ao `call`. No arquivo `.o` esse deslocamento aparece zerado, pois o endereço de `add` só é conhecido na linkedição. Como o código foi copiado manualmente para um *array*, o próprio programa precisa fazer esse papel do linkeditor.

A instrução `call` começa na posição 4 do *array* e possui 5 bytes, logo a próxima instrução está na posição $4 + 5 = 9$. O deslocamento é:

$$\text{desl} = \text{endereço de } \texttt{add} - (\texttt{codigo2} + 9) \tag{1}$$

Esse valor é um inteiro de 32 bits com sinal, gravado nas posições 5 a 8 do *array* em *little-endian* (byte menos significativo primeiro).

```c
int add (int x) {
  return x+1;
}

static unsigned char codigo2[] = {0x55, 0x48, 0x89, 0xe5,
```

```
                                    0xe8, 0x00, 0x00, 0x00, 0x00,
                                    0xc9, 0xc3};

int desl = (int)((long)add - (long)(codigo2 + 9));
codigo2[5] = desl & 0xff;           /* little-endian */
codigo2[6] = (desl >> 8) & 0xff;
codigo2[7] = (desl >> 16) & 0xff;
codigo2[8] = (desl >> 24) & 0xff;

execpage(codigo2, sizeof codigo2);
funcp f2 = (funcp)codigo2;
i = (*f2)(10);
```

A saída foi `foo(10) = 11`: o `call` corrigido chegou corretamente a `add`.

4 Desvio Incondicional (`jmp add`)

Na terceira etapa, a instrução `call add` foi substituída por `jmp add`. Como a chamada passa a ser a última ação da função (*tail call*), o quadro de pilha deixa de ser necessário e o `ret` é executado pela própria `add`, que retorna diretamente ao chamador original de `foo`.

4.1 Código em Assembly (`foo_jmp.s`)

```
.text
.globl foo
foo:
    jmp add
```

4.2 Código de Máquina e Deslocamento

```
0000000000000000 <foo>:
   0:   e9 00 00 00 00        jmp    5 <foo+0x5>
```

O byte `e9` é o *opcode* do `jmp` com deslocamento de 32 bits. O deslocamento segue a mesma regra do `call`: diferença entre o destino e o endereço da próxima instrução, que aqui está na posição $0 + 5 = 5$.

```
static unsigned char codigo3[] = {0xe9, 0x00, 0x00, 0x00, 0x00};

desl = (int)((long)add - (long)(codigo3 + 5));
codigo3[1] = desl & 0xff;
codigo3[2] = (desl >> 8) & 0xff;
codigo3[3] = (desl >> 16) & 0xff;
codigo3[4] = (desl >> 24) & 0xff;

execpage(codigo3, sizeof codigo3);
funcp f3 = (funcp)codigo3;
i = (*f3)(10);
```

A saída também foi `foo(10) = 11`. A tabela a seguir resume as três versões:

Versão	Bytes (hex)	Tamanho	Resultado
`x+1`	`83 c7 01 89 f8 c3`	6 bytes	11
`call add`	`55 48 89 e5 e8 xx xx xx xx c9 c3`	11 bytes	11
`jmp add`	`e9 xx xx xx xx`	5 bytes	11

Tabela 1: Resumo das funções geradas (`xx` = deslocamento calculado em tempo de execução).

5 Limitação do Alcance do Deslocamento em 64 bits

Durante os testes observou-se que as versões com `call` e `jmp` causavam falha de segmentação quando o *array* era declarado como variável local de `main` (ou seja, na pilha), enquanto a versão simples funcionava normalmente.

A causa é o tamanho do deslocamento: as formas `e8` e `e9` usam um inteiro de 32 bits com sinal, que alcança apenas cerca de ± 2 GB a partir da próxima instrução. Em x86-64, a pilha fica em endereços muito altos, distantes da região de código onde está `add`, de modo que a diferença entre os endereços não cabe em 32 bits e o valor gravado fica truncado, desviando para um endereço inválido.

A solução adotada foi declarar `codigo2` e `codigo3` como `static`, de modo que ficam na área de dados do próprio executável, próxima de `add`, e então tornar essas páginas executáveis com `execpage`. A versão simples (`codigo1`) não possui endereços relativos e continua como variável local. Em uma compilação de 32 bits (`-m32`) o problema não ocorreria, pois todo o espaço de endereçamento cabe em um deslocamento de 32 bits.

Registrou-se também que, neste ambiente, apenas a opção `-Wa,-execstack` não foi suficiente para permitir a execução do código, sendo necessário o uso de `execpage` (`mprotect` com `PROT_EXEC`), alternativa prevista no enunciado.

6 Conclusão

O laboratório mostrou na prática que o código de máquina é apenas uma sequência de bytes que a CPU interpreta como instruções, e que o endereço de um *array* pode ser tratado como o endereço de uma função. Foi possível observar o papel do linkeditor ao se corrigir manualmente o deslocamento de `call` e `jmp`, compreendendo que esses desvios são relativos ao endereço da instrução seguinte e que são armazenados em *little-endian*. Também ficou evidente a diferença entre `call` e `jmp` (este não empilha endereço de retorno), o limite de ± 2 GB dos deslocamentos de 32 bits em x86-64 e o papel das proteções do sistema operacional, como o bit NX (*No-Execute*), que exigem marcar explicitamente a memória como executável.